

SICP Study with Emacs - Setup Guide

This guide shows you how to evaluate Scheme code in Emacs while studying Structure and Interpretation of Computer Programs (SICP).

Quick Start

Option 1: Geiser (Recommended)

Installation:

```
M-x package-install RET geiser RET  
M-x package-install RET geiser-mit RET
```

Start REPL:

```
M-x run-geiser
```

Select MIT Scheme when prompted.

Evaluation Commands:

- `(C-x C-e)` - Evaluate expression before cursor
- `(C-c C-e)` - Evaluate last sexp
- `(C-c C-r)` - Evaluate region (selected lines)
- `(C-c C-b)` - Evaluate entire buffer
- `(C-c C-z)` - Switch between code and REPL

Option 2: Built-in run-scheme

No installation needed:

```
M-x run-scheme RET mit-scheme RET
```

Evaluation Commands:

- `(C-x C-e)` - Evaluate expression before cursor
- `(C-c C-r)` - Send region to REPL
- `(C-c C-l)` - Load file into REPL

Practical Workflow Example

1. **Create a Scheme file:**

C-x C-f sicp-exercises.scm

2. Write some code:

scheme

```
(define (square x) (* x x))
```

```
(define (sum-of-squares x y)  
  (+ (square x) (square y)))
```

```
(sum-of-squares 3 4)
```

3. Start your REPL:

M-x run-geiser

4. Switch back to your file:

C-x b sicp-exercises.scm

5. Evaluate line by line:

- Position cursor after `(define (square x) (* x x))`
- Press `C-x C-e`
- Move to next definition and repeat
- Or select multiple lines (`C-SPC` to mark, move cursor) and press `C-c C-r`

Comparison: Geiser vs run-scheme

Geiser (Recommended for SICP)

Pros:

- Rich REPL integration with semantic understanding
- Autodoc shows procedure signatures
- Jump to definition support
- Better error messages with context
- Supports multiple Scheme implementations
- Code completion and introspection

Cons:

- Requires package installation
- Slightly more setup

run-scheme (Built-in)

Pros:

- No installation needed
- Simple and lightweight
- Quick setup

Cons:

- Basic text-based interaction only
- No autodoc or completion
- Limited error navigation
- Minimal code introspection

Tips for SICP Study

- Use **`*scratch*`** buffer for quick tests (default Emacs Lisp, but you can switch mode)
- Create separate **`.scm`** files for each chapter/section
- Keep **REPL visible** alongside your code buffer (`(C-x 2)` to split window)
- Evaluate **incrementally** as you build up definitions
- Use **`C-c C-b`** to reload entire buffer when making changes

Common Key Bindings Summary

Command	Geiser	run-scheme
Eval before cursor	<code>C-x C-e</code>	<code>C-x C-e</code>
Eval region	<code>C-c C-r</code>	<code>C-c C-r</code>
Eval buffer	<code>C-c C-b</code>	<code>C-c C-l</code>
Switch to REPL	<code>C-c C-z</code>	<code>C-c C-z</code>
Eval last sexp	<code>C-c C-e</code>	N/A

Getting Help

- `C-h k` followed by a key sequence shows what that key does
- `C-h m` shows all commands available in current mode
- In Geiser REPL: `C-c C-d C-d` on a symbol shows documentation

Recommendation

For SICP study, **use Geiser** for the best development experience. The inline documentation and better error messages are invaluable when working through exercises.

If you prefer minimal setup or are just testing quickly, **run-scheme works fine** and gets the job done.

Happy hacking!